```
if defined?(b) && b
  # if b is already defined and neither nil nor false:
  b
else
  # if b is undefined, nil or false:
  b = calculate_initial_value
end
```

If you look at this if-based translation of a Nil Guard, you will see that Nil Guards are unable to distinguish false from nil. In our previous example, b is false, so the Nil Guard reinitializes it every time.

This little wrinkle of Nil Guards usually goes unnoticed, but it can also cause the occasional hard-to-spot bug. For this reason, you shouldn't use Nil Guards to initialize variables that can have false (or nil, for that matter) as a legitimate value.

Self Yield

When you pass a block to a method, you expect the method to call back to the block with yield. A twist on callbacks is that an object can also pass *itself* to the block. Let's see how this can be useful.

The Faraday Example

In the Faraday HTTP library, you typically initialize an HTTP connection with a URL and a block:

common_idioms/faraday_example.rb

```
require 'faraday'

conn = Faraday.new("https://twitter.com/search") do |faraday|
  faraday.response      :logger
  faraday.adapter       Faraday.default_adapter
  faraday.params["q"]   = "ruby"
  faraday.params["src"] = "typd"
end

response = conn.get
response.status           # => 200
```

This code sets the parameters for the connection. If you wish, you can get the same results by passing a hash of parameters to Faraday.new—but the block-based style has the advantage of making it clear that all the statements in the block are focusing on the same object. If you like this style, you might want to peek inside Faraday's source code and see how it is implemented. Faraday.new actually creates and returns a Faraday::Connection object: